

Milestone 1

Group Members-

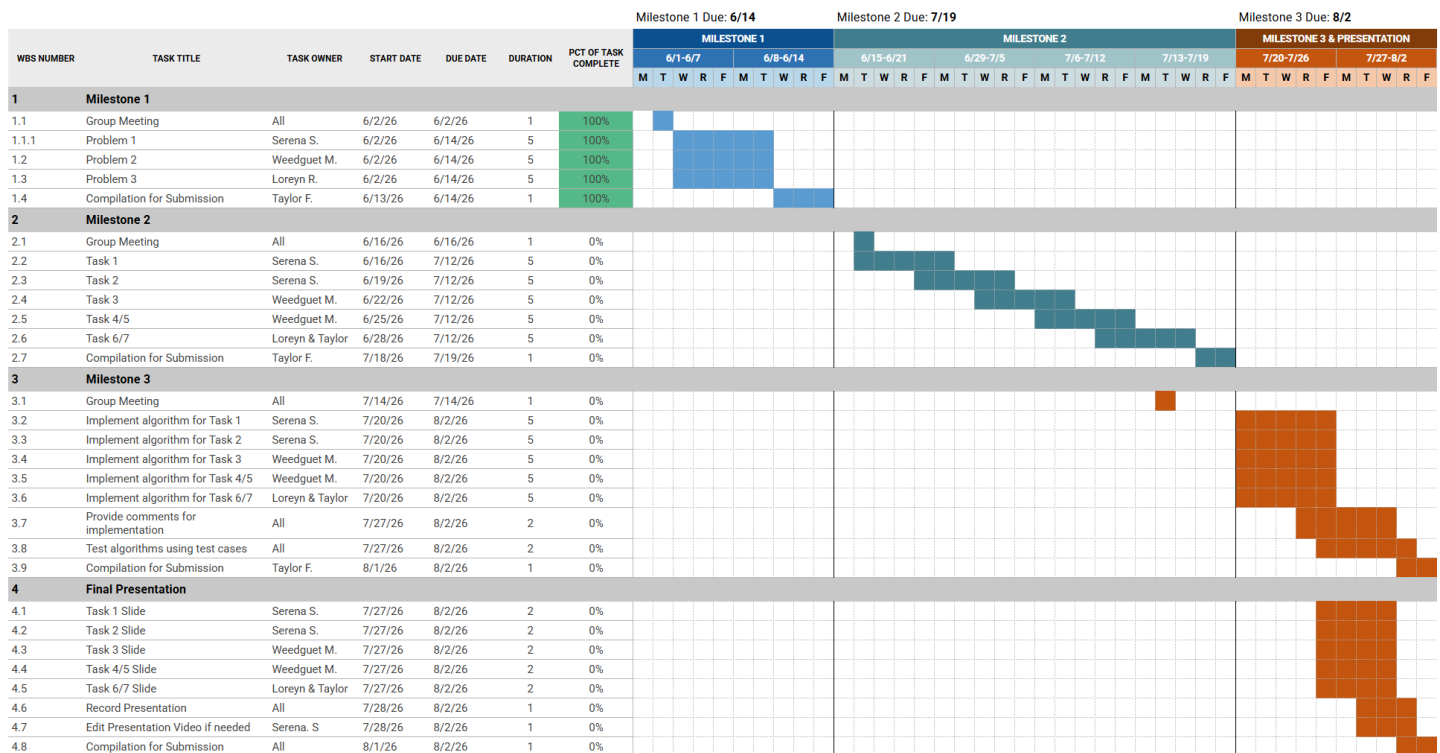
- Taylor Filson (tyler.filson@ufl.edu, GitHub: TFilson)
- Weedguet Mildort (weedguet.mildort@ufl.edu, GitHub: weedguetmildort)
- Loreyn Reid (l.reid1@ufl.edu, GitHub: loreyn626)
- Serena Seymour (serenaseymour@ufl.edu, GitHub: serenaseymour)

Member Roles-

- Taylor Filson: Development, Time Keeper
- Weedguet Mildort: Development
- Loreyn Reid: Development
- Serena Seymour: Development

Communication Method- Discord

Project Gantt Chart-



GitHub Repo- <https://github.com/serenaseymour/COP4533-Group-1-Summer-2026>

Solutions-

Problem Statement 1:

$$A = \begin{matrix} & \begin{matrix} \text{Day 1} & \text{Day 2} & \text{Day 3} & \text{Day 4} & \text{Day 5} \end{matrix} \\ \begin{matrix} \text{Stock 1} \\ \text{Stock 2} \\ \text{Stock 3} \\ \text{Stock 4} \end{matrix} & \begin{bmatrix} 12 & 1 & 5 & 3 & 16 \\ 4 & 4 & 13 & 4 & 9 \\ 6 & 8 & 6 & 1 & 2 \\ 14 & 3 & 4 & 8 & 10 \end{bmatrix} \end{matrix}$$

Stock 1				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	1-12	-11	Loss
1	3	5-12	-7	Loss
1	4	3-12	-9	Loss
1	5	16-12	4	Good
2	3	5-1	4	Good
2	4	3-1	2	Good
2	5	16-1	15	Best
3	4	3-5	-2	Loss
3	5	16-5	11	Good
4	5	16-3	13	Good

Stock 2				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	4-4	0	Even
1	3	13-4	9	Best
1	4	4-4	0	Even
1	5	9-4	5	Good
2	3	13-4	9	Best
2	4	4-4	0	Even
2	5	9-4	5	Good
3	4	13-4	9	Best
3	5	13-9	5	Good
4	5	4-9	-5	Loss

Stock 3				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	8-6	2	Best
1	3	6-6	0	Even
1	4	1-6	-5	Loss
1	5	2-6	-4	Loss
2	3	6-8	-2	Loss
2	4	1-8	-7	Loss
2	5	2-8	-6	Loss
3	4	1-6	-5	Loss
3	5	2-6	-4	Loss
4	5	2-1	-1	Loss

Stock 4				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	3-14	-11	Loss
1	3	4-14	-10	Loss
1	4	8-14	-6	Loss
1	5	10-14	-4	Loss
2	3	4-3	1	Good
2	4	8-3	5	Good
2	5	10-3	7	Best
3	4	8-4	4	Good
3	5	10-4	6	Good
4	5	10-8	2	Good

Stock 1 Best: **15**

Stock 2 Best: 9

Stock 3 Best: 2

Stock 4 Best: 7

Max Profit= 15
(1, 2, 5, 15)

Stock 1, bought on day 2 (1), sold on day 5 (16), profit= 15

Problem Statement 2:

$$A = \begin{matrix} & \begin{matrix} \text{Day 1} & \text{Day 2} & \text{Day 3} & \text{Day 4} & \text{Day 5} \end{matrix} \\ \begin{matrix} \text{Stock 1} \\ \text{Stock 2} \\ \text{Stock 3} \\ \text{Stock 4} \end{matrix} & \begin{bmatrix} 25 & 30 & 15 & 40 & 50 \\ 10 & 20 & 30 & 25 & 5 \\ 30 & 45 & 35 & 10 & 15 \\ 5 & 50 & 35 & 25 & 45 \end{bmatrix} \end{matrix}$$

$$k = 3$$

Stock 1					Stock 2				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	30-25	5	Gain	1	2	20-10	10	Gain
1	3	15-25	-10	Loss	1	3	30-10	20	Best
1	4	40-25	15	Gain	1	4	25-10	15	Gain
1	5	50-25	25	Gain	1	5	5-10	-5	Loss
2	3	15-30	-15	Loss	2	3	30-20	10	Gain
2	4	40-30	10	Gain	2	4	25-20	5	Gain
2	5	50-30	20	Gain	2	5	5-20	-15	Loss
3	4	40-15	25	Gain	3	4	25-30	-5	Loss
3	5	50-15	35	Best	3	5	5-30	-25	Loss
4	5	50-40	10	Gain	4	5	5-25	-20	Loss

Stock 3					Stock 4				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	45-30	15	Best	1	2	50-5	45	Best
1	3	35-30	5	Gain	1	3	35-5	30	Gain
1	4	10-30	-20	Loss	1	4	25-5	20	Gain
1	5	15-30	-15	Loss	1	5	45-5	40	Gain
2	3	35-45	-10	Loss	2	3	35-50	-15	Loss
2	4	10-45	-35	Loss	2	4	25-50	-25	Loss
2	5	15-45	-30	Loss	2	5	45-50	-5	Loss
3	4	10-35	-25	Loss	3	4	25-35	-10	Loss
3	5	15-35	-20	Loss	3	5	45-35	10	Gain
4	5	15-10	5	Gain	4	5	45-25	20	Gain

All the best profits:

Stock 1 Best: 35

Stock 2 Best: 20

Stock 3 Best: 15

Stock 4 Best: 45

The top two best profits are 35 and 45, and they are acquired between day 1 to day 2, and between day 3 to day 5, which leaves day 2 and day 3 available to take another transaction.

All the results between day 2 and 3:

Stock 1 Best: -15

Stock 2 Best: 10

Stock 3 Best: -10

Stock 4 Best: -15

The best result between day 2 and day 3 is stock 2.

A sequence of at most k transactions, each involving the purchase and sale of a single stock, that yields the maximum profit is: [(4, 1, 2),(2, 2, 3),(1, 3, 5)]

Stock 4, bought on day 1 (5), sold on day 2 (50), profit= 45

Stock 2, bought on day 2 (20), sold on day 3 (30), profit= 10

Stock 1, bought on day 3 (15), sold on day 5 (50), profit= 35

Hence the total maximum possible profit is 90. To reach the maximum possible profit, the first step was to identify the two best transactions, then determine the best third transaction that satisfied all constraints.

Problem Statement 3:

$$A = \begin{matrix} & \begin{matrix} \text{Day 1} & \text{Day 2} & \text{Day 3} & \text{Day 4} & \text{Day 5} & \text{Day 6} & \text{Day 7} \end{matrix} \\ \begin{matrix} \text{Stock 1} \\ \text{Stock 2} \\ \text{Stock 3} \\ \text{Stock 4} \\ \text{Stock 5} \end{matrix} & \begin{bmatrix} 7 & 1 & 5 & 3 & 6 & 8 & 9 \\ 2 & 4 & 3 & 7 & 9 & 1 & 8 \\ 5 & 8 & 9 & 1 & 2 & 3 & 10 \\ 9 & 3 & 4 & 8 & 7 & 4 & 1 \\ 3 & 1 & 5 & 8 & 9 & 6 & 4 \end{bmatrix} \end{matrix} \quad c = 2$$

Based on our current example of 7 consecutive days and the 2 day trading restriction, the max number of full transactions (buy and sell) is 2. To be able to make a second full transaction, the sell day of transaction 1 can be no later than day 3. The purchase date of transaction 2 can be no later than day 6.

T1: Transaction 1 Results

Note: Eligibility to buy again is based on the statement above. If the sell day of transaction 1 is after day 3, then for calculation purposes, it is marked not eligible to buy again. There would not be enough time to make a second full transaction given the trading restriction.

Best E refers to the highest profit of those eligible to buy again. Best O is the overall highest profit.

Stock 1					
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Eligible to Buy Again (E)
1	2	1-2	-6	Loss	Y - On Days 5 or 6
1	3	2-3	-2	Loss	Y - On Day 6
1	4	3-4	-4	Loss	N
1	5	4-5	-1	Loss	N
1	6	5-6	1	Gain	N
1	7	6-7	2	Gain	N
2	3	1-2	4	Gain (Best E)	Y - On Day 6
2	4	2-3	2	Gain	N
2	5	3-4	5	Gain	N
2	6	4-5	7	Gain	N
2	7	5-6	8	Gain (Best O)	N
3	4	1-2	-2	Loss	N
3	5	2-3	1	Gain	N
3	6	3-4	3	Gain	N
3	7	4-5	4	Gain	N
4	5	1-2	3	Gain	N
4	6	2-3	5	Gain	N
4	7	3-4	6	Gain	N
5	6	1-2	2	Gain	N
5	7	2-3	3	Gain	N
6	7	1-2	1	Gain	N

COP4533
Summer 2026
Group 1

Stock 2					
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Eligible to Buy Again (E)
1	2	4-2	2	Gain (Best E)	Y - On Days 5 or 6
1	3	3-2	1	Gain	Y - On Day 6
1	4	7-2	5	Gain	N
1	5	9-2	7	Gain (Best O)	N
1	6	1-2	-1	Loss	N
1	7	8-2	6	Gain	N
2	3	3-4	-1	Loss	Y - On Day 6
2	4	7-4	3	Gain	N
2	5	9-4	5	Gain	N
2	6	1-4	-3	Loss	N
2	7	8-4	4	Gain	N
3	4	7-3	4	Gain	N
3	5	9-3	6	Gain	N
3	6	1-3	-2	Loss	N
3	7	8-3	5	Gain	N
4	5	9-7	2	Gain	N
4	6	1-7	-6	Loss	N
4	7	8-7	1	Gain	N
5	6	1-9	-8	Loss	N
5	7	8-9	-1	Loss	N
6	7	8-1	7	Gain (Best O)	N

Stock 3					
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Eligible to Buy Again (E)
1	2	8-5	3	Gain	Y - On Days 5 or 6
1	3	9-5	4	Gain (Best E)	Y - On Day 6
1	4	1-5	-4	Loss	N
1	5	2-5	-3	Loss	N
1	6	3-5	-2	Loss	N
1	7	10-5	5	Gain	N
2	3	9-8	1	Gain	Y - On Day 6
2	4	1-8	-7	Loss	N
2	5	2-8	-6	Loss	N
2	6	3-8	-5	Loss	N
2	7	10-8	2	Gain	N
3	4	1-9	-8	Loss	N
3	5	2-9	-7	Loss	N
3	6	3-9	-6	Loss	N
3	7	10-9	1	Gain	N
4	5	2-1	1	Gain	N
4	6	3-1	2	Gain	N
4	7	10-1	9	Gain (Best O)	N
5	6	3-2	1	Gain	N
5	7	10-2	8	Gain	N
6	7	10-3	7	Gain	N

COP4533
Summer 2026
Group 1

Stock 4						
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Eligible to Buy Again (E)	
1	2	3-9	-6	Loss	Y - On Days 5 or 6	
1	3	4-9	-5	Loss	Y - On Day 6	
1	4	8-9	-1	Loss	N	
1	5	7-9	-2	Loss	N	
1	6	4-9	-5	Loss	N	
1	7	1-9	-8	Loss	N	
2	3	4-3	1	Gain (Best E)	Y - On Day 6	
2	4	8-3	5	Gain (Best O)	N	
2	5	7-3	4	Gain	N	
2	6	4-3	1	Gain	N	
2	7	1-3	-2	Loss	N	
3	4	8-4	4	Gain	N	
3	5	7-4	3	Gain	N	
3	6	4-4	0	Gain	N	
3	7	1-4	-3	Loss	N	
4	5	7-8	-1	Loss	N	
4	6	4-8	-4	Loss	N	
4	7	1-8	-7	Loss	N	
5	6	4-7	-3	Loss	N	
5	7	1-7	-6	Loss	N	
6	7	1-4	-3	Loss	N	

Stock 5						
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Eligible to Buy Again (E)	
1	2	1-3	-2	Loss	Y - On Days 5 or 6	
1	3	5-3	2	Gain	Y - On Day 6	
1	4	8-3	5	Gain	N	
1	5	9-3	6	Gain	N	
1	6	6-3	3	Gain	N	
1	7	4-3	1	Gain	N	
2	3	5-1	4	Gain (Best E)	Y - On Day 6	
2	4	8-1	7	Gain	N	
2	5	9-1	8	Gain (Best O)	N	
2	6	6-1	5	Gain	N	
2	7	4-1	3	Gain	N	
3	4	8-5	3	Gain	N	
3	5	9-5	4	Gain	N	
3	6	6-5	1	Gain	N	
3	7	4-5	-1	Loss	N	
4	5	9-8	1	Gain	N	
4	6	6-8	-2	Loss	N	
4	7	4-8	-4	Loss	N	
5	6	6-9	-3	Loss	N	
5	7	4-9	-5	Loss	N	
6	7	4-6	-2	Loss	N	

T2: Transaction 2 Results & Final Profits

For each transaction that was eligible for a second transaction, we compute the total profit of each combination of T1 and T2 to find the max profit.

Note: The Max Profit takes into account single transactions from the results above and the total profits from the 2nd transaction results below.

Transaction 2 (Stock 1 Transaction 1)						
Stock	Buy Day (j1)	Sell Day (j2)	Profit	Total Profit, T1: j1 = 1, j2 = 2, (-6 + profit)	Total Profit T1: j1 = 1, j2 = 3, (-2 + profit)	Total Profit T1: j1 = 2, j3 = 3, (4 + profit)
1	5	6	2	-4		
1	5	7	3	-3		
1	6	7	1	-5	-1	5
2	5	6	-8	-14		
2	5	7	-1	-7		
2	6	7	7	1	5	11
3	5	6	1	-5		
3	5	7	8	2		
3	6	7	7	1	5	11
4	5	6	-3	-9		
4	5	7	-6	-12		
4	6	7	-3	-9	-5	1
5	5	6	-3	-9		
5	5	7	-5	-11		
5	6	7	-2	-8	-4	2

Stock 1:

Max Profit = 11

[(1, 2, 3), (2,6,7)]

[(1, 2, 3), (3,6,7)]

Transaction 2 (Stock 2 Transaction 1)						
Stock	Buy Day (j1)	Sell Day (j2)	Profit	Total Profit, T1: j1 = 1, j2 = 2, (2 + profit)	Total Profit T1: j1 = 1, j2 = 3, (1 + profit)	Total Profit T1: j1 = 2, j3 = 3, (-1 + profit)
1	5	6	2	4		
1	5	7	3	5		
1	6	7	1	3	2	0
2	5	6	-8	-6		
2	5	7	-1	1		
2	6	7	7	9	8	6
3	5	6	1	3		
3	5	7	8	10		
3	6	7	7	9	8	6
4	5	6	-3	-1		
4	5	7	-6	-4		
4	6	7	-3	-1	-2	-4
5	5	6	-3	-1		
5	5	7	-5	-3		
5	6	7	-2	0	-1	-3

Stock 2:

Max Profit = 10

[(2, 1, 2), (3,5,7)]

Transaction 2 (Stock 3 Transaction 1)						
Stock	Buy Day (j1)	Sell Day (j2)	Profit	Total Profit, T1: j1 = 1, j2 = 2, (3 + profit)	Total Profit T1: j1 = 1, j2 = 3, (4 + profit)	Total Profit T1: j1 = 2, j3 = 3, (1 + profit)
1	5	6	2	5		
1	5	7	3	6		
1	6	7	1	4	5	2
2	5	6	-8	-5		
2	5	7	-1	2		
2	6	7	7	10	11	8
3	5	6	1	4		
3	5	7	8	11		
3	6	7	7	10	11	8
4	5	6	-3	0		
4	5	7	-6	-3		
4	6	7	-3	0	1	-2
5	5	6	-3	0		
5	5	7	-5	-2		
5	6	7	-2	1	2	-1

Stock 3:

Max Profit = 11

[(3, 1, 2), (3,5,7)]

[(3, 1, 3), (2,6,7)]

[(1, 1, 3), (3,6,7)]

Transaction 2 (Stock 4 Transaction 1)						
Stock	Buy Day (j1)	Sell Day (j2)	Profit	Total Profit, T1: j1 = 1, j2 = 2, (-6 + profit)	Total Profit T1: j1 = 1, j2 = 3, (-5 + profit)	Total Profit T1: j1 = 2, j3 = 3, (1 + profit)
1	5	6	2	-4		
1	5	7	3	-3		
1	6	7	1	-5	-4	2
2	5	6	-8	-14		
2	5	7	-1	-7		
2	6	7	7	1	2	8
3	5	6	1	-5		
3	5	7	8	2		
3	6	7	7	1	2	8
4	5	6	-3	-9		
4	5	7	-6	-12		
4	6	7	-3	-9	-8	-2
5	5	6	-3	-9		
5	5	7	-5	-11		
5	6	7	-2	-8	-7	-1

Stock 4:

Max Profit = 8

[(4, 2, 3), (2,6,7)]

[(4, 2, 3), (3,6,7)]

Transaction 2 (Stock 5 Transaction 1)						
Stock	Buy Day (j1)	Sell Day (j2)	Profit	Total Profit, T1: j1 = 1, j2 = 2, (-2 + profit)	Total Profit T1: j1 = 1, j2 = 3, (2 + profit)	Total Profit T1: j1 = 2, j3 = 3, (4 + profit)
1	5	6	2	0		
1	5	7	3	1		
1	6	7	1	-1	3	5
2	5	6	-8	-10		
2	5	7	-1	-3		
2	6	7	7	5	9	11
3	5	6	1	-1		
3	5	7	8	6		
3	6	7	7	5	9	11
4	5	6	-3	-5		
4	5	7	-6	-8		
4	6	7	-3	-5	-1	1
5	5	6	-3	-5		
5	5	7	-5	-7		
5	6	7	-2	-4	0	2

Stock 5:

Max Profit = 11

[(5, 2, 3), (2,6,7)]

[(5, 2, 3), (3,6,7)]

The overall max profit is 11. This could be obtained through multiple transactions. All are listed below.

[(1, 2, 3), (2,6,7)]

[(1, 2, 3), (3,6,7)]

[(3, 1, 2), (3,5,7)]

[(3, 1, 3), (2,6,7)]

[(1, 1, 3), (3,6,7)]

[(5, 2, 3), (2,6,7)]

[(5, 2, 3), (3,6,7)]

We only need to output one of these results:

Output: [(1, 2, 3), (2,6,7)]

Milestone 2

Programming Language: Python

Task 1: Brute force algorithm for Problem 1 in $O(m \cdot n^2)$

```
function BruteForce(A, m, n):  
    // A is 0-based 2D array of size m x n  
    max_profit = -1  
    best_stock = 0  
    best_buy = 0  
    best_sell = 0  
  
    for i = 1 to m:           // You go through every single stock one by one. m is the number of rows (stocks)  
        for j1 = 1 to n:      // For the current stock, you try every possible day as a buy day. j1 is the buy day, n is number of  
                               // columns (days)  
            for j2 = j1 + 1 to n: // For each buy day chosen, try every later day as a sell day, ensuring you only sell after buying  
                                   // (j1 < j2). This inner loop is what makes it  $O(n^2)$  per stock  
                profit = A[i-1][j2-1] - A[i-1][j1-1] // Calculate profit. You subtract 1 from indices because arrays in most  
                                                         // programming languages are 0-based. This gives sell price minus buy price  
                if profit > max_profit: // If this transaction is better than anything you've seen across all stocks and all pairs, the  
                                         // algorithm remembers it. Update the best profit and the exact stock/buy/sell days  
                    max_profit = profit  
                    best_stock = i  
                    best_buy = j1  
                    best_sell = j2  
  
    if max_profit > 0: // Final check. If you found any profitable transaction, return the best one. Otherwise return (0,0,0,0)  
        return (best_stock, best_buy, best_sell, max_profit)  
    else:  
        return (0, 0, 0, 0)
```

Time complexity: m stocks $\times$ n buy days $\times$ $\sim n$ sell days = $O(m \cdot n^2)$

Task 2: Greedy algorithm for Problem 1 in $O(m \cdot n)$

```
function Greedy(A, m, n):
    // A is 0-based 2D array of size m x n
    max_profit = -1
    best_stock = 0
    best_buy = 0
    best_sell = 0

    for i = 1 to m:           // Same as brute force, go through every stock one by one. m is the number of rows (stocks)
        if n < 2:             // If there are less than 2 days, you can't do a transaction. Skip this stock
            continue

        min_price = A[i-1][0] // Start with the price on day 1 as the minimum (best buy so far). Record that you would buy
                               // on day 1
        min_day = 1
        local_max_profit = 0
        local_buy = 1
        local_sell = 1

        for j = 2 to n:       // Scan every day starting from day 2. This is only one pass per stock, that's why it's O(n)
            current_profit = A[i-1][j-1] - min_price // If you sell today (day j), what profit would you get using the cheapest
                                                    // price seen before today?

            if current_profit > local_max_profit:    // If selling today gives a better profit than any previous sell opportunity
                                                    // on this stock, remember this profit and the days

                local_max_profit = current_profit
                local_buy = min_day
                local_sell = j

            if A[i-1][j-1] < min_price:             // If today's price is cheaper than the previous minimum, update the minimum price
                                                    // and the buy day. This prepares for future potential sell days

                min_price = A[i-1][j-1]
                min_day = j

        if local_max_profit > max_profit:           // After finishing all days for this stock, check if this stock's best transaction is
                                                    // better than the best you've seen from other stocks. If yes, update the global
                                                    // best

            max_profit = local_max_profit
            best_stock = i
            best_buy = local_buy
            best_sell = local_sell

    if max_profit > 0:
        return (best_stock, best_buy, best_sell, max_profit)
    else:
        return (0, 0, 0, 0)
```

Time complexity: $m \text{ stocks} \times n \text{ days} = O(m \cdot n)$

Task 3: Dynamic Algorithm for Problem 1 in $O(m \cdot n)$

To solve this problem dynamically in $O(m \cdot n)$ time we will keep a table that tracks the minimum priced day before each possible sell date. We will do so with 3 separate $n \cdot m$ loops.

This solution is similar to the greedy approach though technically runs worse. This is an example of a problem that is best solved by a greedy approach (especially considering the space complexity that is cut out by not employing dynamic programming for this problem).

The recursive subproblem here is that in order to calculate the max profit, we first need to find the minimum priced day from the days before to buy each respective stock.

$M[i, j]$ stores the index of the minimum buy price before any given day for stock i on day j .

function dynamicOne(A, m, n):

STEP ONE:

Create a two-dimensional array M with dimensions $m \times n$, with each value initialized to infinity.

STEP TWO:

For each i to m //iterate through each stock (by index)

For each j (starting at the starting index + 1) to n //iterate through each day for each stock, skipping the first day of the time period.

if ($A[i, j-1] \leq A[i, M[i, j-1]]$): //if the value of the day prior is less than the minimum sell price up to that day,

$M[i, j] = j-1$ //set the minimum buy price before this day j to the index of the day prior

Else:

$M[i, j] = M[i, j-1]$ //Otherwise, retain the minimum buy price of the day before.

STEP THREE:

tempMax = 0 //pure max profit

tempBuy = 0 //index of when to buy

tempSell = 0 //index of when to sell

tempStock = 0 //index of which stock we're buying/selling

For each i to m :

For each j (starting at the starting index + 1) to n

$\text{maxProfitForDay} = A[i, j] - A[i, M[i, j]]$ //The max profit we can get by selling on any day is going to be the price today minus the price on the minimum day before this one for this stock.

if maxProfitForDay > tempMax: *//keeping track of max alongside each of the values that we want to return by the end.*

tempMax = maxProfitForDay

tempBuy = M[i, j]

tempSell = j

tempStock = i

if tempMax < 0:

return (0, 0, 0, 0)

else:

return (tempStock, tempBuy, tempSell, tempMax)

This algorithm will find the maximum profit transaction in $O(n*m)$ time.

Task 5: Dynamic Programming Algorithm for Problem 2 in $O(m \cdot n \cdot k)$

```
function DynamicProgramming(A, m, n, k):
    // A is a 0-based 2D array of size m x n, where m is the number of stocks and n is the number of days
    // k is the maximum number of transactions allowed
    if n < 2 or k == 0:
        return []                                     // If there are fewer than 2 days or no transactions allowed,
                                                    // no profitable buy-sell transaction can be made

    create DP[0...k][1...n]                          // Create a 2D table named DP
    create Choice[0...k][1...n]                      // Create another 2D table named Choice

    // DP[t][d] represents the maximum profit possible using at most t transactions
    // considering only days 1 through d
    for d = 1 to n:
        DP[0][d] = 0                                // With 0 transactions, profit is always 0
        Choice[0][d] = None

    for t = 1 to k:
        DP[t][1] = 0                                // On day 1, we cannot sell yet, so profit is 0
        Choice[t][1] = None

    for t = 1 to k:                                  // Try using up to t transactions
        for i = 1 to m:
            best_buy_profit[i] = DP[t-1][1] - A[i-1][0]
            best_buy_day[i] = 1                      // Start by assuming stock i is bought on day 1
                                                    // best_value keeps track of the best previous profit
                                                    // minus the price paid to buy stock i

        for d = 2 to n:                              // Scan days from left to right
            DP[t][d] = DP[t][d-1]
            Choice[t][d] = ("skip", t, d-1)          // First assume we do not sell anything on day d
                                                    // This carries forward the best profit from yesterday

            for i = 1 to m:                            // Try selling each stock on day d
                current_profit = best_buy_profit[i] + A[i-1][d-1]
                                                    // If we sell stock i today, this gives the total profit:
                                                    // previous best profit before/at the buy day
                                                    // plus today's selling price
                                                    // minus the buying price already stored in best_value
                if current_profit > DP[t][d]:
                    DP[t][d] = current_profit
                    Choice[t][d] = ("sell", i, best_buy_day[i], d)
                                                    // If selling stock i today gives a better profit,
                                                    // remember the stock, buy day, and sell day

            for i = 1 to m:                            // Update the best possible buy option for future days
                possible_buy_profit = DP[t-1][d] - A[i-1][d-1]
                                                    // This represents buying stock i on day d
                                                    // after completing at most t-1 transactions by day d
                if possible_buy_profit > best_buy_profit[i]:
                    best_buy_profit[i] = possible_buy_profit
                    best_buy_day[i] = d              // If buying today gives a better future opportunity,
                                                    // update the best buy day for this stock

    if DP[k][n] <= 0:
        return []                                     // If no positive profit is possible, return empty sequence

    result = BuildSolution(Choice, k, n)             // Helper function
    return result
```

Time complexity: m stocks $\times$ n days $\times$ k transactions = $O(m \cdot n \cdot k)$

Task 5 helper function:

```
function BuildSolution(Choice, t, d):  
  
    if t == 0 or d <= 1:  
        return [] // No transactions left or no possible sell day  
  
    if Choice[t][d] == None:  
        return []  
  
    if Choice[t][d].type == "skip":  
        return BuildSolution(Choice, t, d-1) // If no transaction ends on day d,  
                                              // move back to the previous day  
  
    else:  
        stock = Choice[t][d].stock  
        buy_day = Choice[t][d].buy_day  
        sell_day = Choice[t][d].sell_day  
  
        previous_transactions = BuildSolution(Choice, t-1, buy_day)  
        // Since this transaction buys on buy_day,  
        // earlier transactions must finish on or before buy_day.  
        // This matches the non-overlapping rule.  
  
        return previous_transactions + [(stock, buy_day, sell_day)]
```


Task 6: Dynamic Programming Algorithm for Problem 3 – $O(m \cdot n^2)$

Let's keep track of the total max profit possible by the end of day, d . To do this, we will focus on two cases. Case 1 is to do nothing on day, d . Case 2 is to sell on day d . If we choose to sell, then we will determine the best day to buy considering the max profit that can be obtained by combining any previous transactions. It is important to remember that if there is a previous transaction, its sell date must be on or before day $j-c-1$ (j is the current buy date we are considering).

To solve this problem dynamically in $O(m \cdot n^2)$, we used nested loops. First, we iterate through n days, then we iterate through m stocks, and then we iterate through up to $n-1$ buy days.

Function DynamicProgramming(A, m, n, c):

STEP ONE:

Create an array DP with dimensions $1 \times n$ where n is the number of days. // this array stores the max profit up to day d

Create an array $Choice$ with dimensions $1 \times n$ where n is the number of days. // this array stores the choices made during each day. This could be a "skip" day in which neither a purchase or sale is made or it could store the details of a transaction (stock, buy day, sell day).

STEP TWO:

if $n < 2$: // if there are less than 2 days, no transactions can be made
return [] // cannot return a sequence if no full transactions are made

for $d = 1$ to n : // for each day

if $d = 1$:

$DP[1] = 0$ // On day 1, you cannot sell since you cannot buy a stock
and sell it on the same day. Even if you do, the total profit is zero.

$Choice[1] = ("skip")$

else:

$DP[d] = DP[d-1]$ // on day 2 or later, first case is that we do not sell

$Choice[d] = ("skip")$

for $i=1$ to m : // for each stock

```
for j = 1 to d-1:           //for each buy day

    if (j-c-1) >= 1:         //ensures that we only check valid days

        current_total_profit = DP[j-c-1] + A[i][d] - A[i][j]

        // test 2nd case which would be choosing to sell today.

        DP[j-c-1] = max profit from previous transactions by the end of day
        (j-c-1). This is to ensure that we are adhering to the waiting period.

        A[i][d] - A[i][j] = the profit from the sale = selling price - buy price.

    else:

        current_total_profit = A[i][d] - A[i][j]

    if current_total_profit > DP[d]:           // if the current_total _profit is greater
                                                than the previously stored profit of DP[d]

        DP[d] = current_total_profit           // update the profit amount

        Choice[d] = (i, j, d)                 // store the choice

result = BuildSolution(Choice, n, c )         // return the sequences that yield the max profit
return result
```

STEP 3:

```
function BuildSolution(Choice, d, c):

    if d <= 1:

        return []

    if Choice[d].type == "skip"

        return BuildSolution(Choice, d-1)           // returns the choice from the previous
                                                        day since there would not be a sequence of a transaction on a skip day

    else:

        stock = Choice[d].stock           // assign variables to the values of Choice[d]

        buy_day = Choice[d].buy_day

        sell_day = Choice[d].sell_day
```

```
previous_transactions = BuildSolution(Choice, buy_day-c-1, c) // pull the  
sequence from the previous transaction if there is one
```

```
return previous_transactions + [(stock, buy_day, sell_day)] // return the  
previous sequences if they exist and the current sequence
```

Time Complexity: $m \text{ stocks} \times n \text{ days} \times n-1 \text{ days} = O(m \cdot n^2)$.